Sistemas Distribuidos

Programación de Sistemas RPC: Google gRPC

Diego Sevilla Ruiz

DITEC
Facultad de Informática

Murcia, 2026

Índice

1. Introducción a gRPC
2. Protocol Buffers versión 3
3. Uso desde Java
4. Buenas prácticas

Introducción a Google gRPC

- gRPC[1] es un servicio RPC construido por Google para la comunicación de servicios en sus aplicaciones
- Consta, al igual que CORBA, de:
  1. Un lenguaje de especificación de tipos e interfaces, basado en **protocol buffers**[2] versión 3
  2. Unos *mappings* a lenguajes de programación
  3. Un programa generador de clientes y servidores
  4. Una codificación por el cable
- Permiten comunicación petición/respuesta o en *stream* (varias respuestas en un flujo)

[1] `http://www.grpc.io/`

[2] `https://developers.google.com/protocol-buffers/`

gRPC

About Docs Showcase Blog Community Search this site...

gRPC

A high performance, open source universal RPC framework

Learn more

Get started!

Go C++ Java Python C# •••

Why gRPC?

gRPC is a modern open source high performance Remote Procedure Call (RPC) framework that can run in any environment. It can efficiently connect services in and across data centers with pluggable support for load balancing, tracing, health checking and authentication. It is also applicable in last mile of distributed computing to connect devices, mobile applications and browsers to backend services.

Simple service definition

Define your service using Protocol Buffers, a powerful binary serialization toolset and language

Start quickly and scale

Install runtime and dev environments with a single line and also scale to millions of RPCs per second with the framework

Works across languages and platforms

Automatically generate idiomatic client and server stubs for your service in a variety of languages and platforms

Bi-directional streaming and integrated auth

Bi-directional streaming and fully integrated pluggable authentication with HTTP/2-based transport

Used by

Square NETFLIX CoreOS Cockroach LABS CISCO JUNIPER NETWORKS

Introducción a Protocol Buffers

- gRPC consta de dos partes:
  - Por un lado, **Protocol Buffers**, que especifica la codificación y la formación de los mensajes y los tipos de datos
  - Por otro lado, **servicios en gRPC** en donde esos tipos de datos se usan para definir servicios que contienen funciones que se pueden llamar de forma remota
- Protocol Buffers: esquema de especificación de tipos y serialización similar a IDL, XML o JSON
- Google eligió un nuevo lenguaje con respecto a XML o JSON porque:
  - Es más sencillo
  - Es de 3 a 10 veces más pequeño
  - Es de 20 a 100 veces más rápido
  - Es menos ambiguo
  - Genera menos clases y tiene un API más sencillo

Introducción a Protocol Buffers II

Protocol buffers are now Google's lingua franca for data – at time of writing, there are 48,162 different message types defined in the Google code tree across 12,183 .proto files. They're used both in RPC systems and for persistent storage of data in a variety of storage systems.

Sintaxis de los archivos `.proto`

- Comienzan con la expresión `syntax = "proto3";`
- Contienen opciones específicas para el lenguaje (Java, C++, etc.)
- Pueden importar a otros ficheros `.proto`
- Contienen especificación de paquetes
- Contienen la definición de tipos (`message`s)
- Contienen la definición de servicios (`service`)

Tipos Básicos

.proto Type	Notes	C++ Type	Java Type	Python Type
`double`	double	`double`	`float`	`float64`
`float`	float	`float`	`float`	`float32`
`int32`	Uses variable-length encoding. Inefficient for encoding negative numbers – if your field is likely to have negative values, use sint32 instead.	`int32`	`int`	`int`
`int64`	Uses variable-length encoding. Inefficient for encoding negative numbers – if your field is likely to have negative values, use sint64 instead.	`int64`	`long`	`int/long`
`uint32`	Uses variable-length encoding.	`uint32`	`int`	`int/long`
`uint64`	Uses variable-length encoding.	`uint64`	`long`	`int/long`
`sint32`	Uses variable-length encoding. Signed int value. These more efficiently encode negative numbers than regular int32s.	`int32`	`int`	`int`
`sint64`	Uses variable-length encoding. Signed int value. These more efficiently encode negative numbers than regular int64s.	`int64`	`long`	`int/long`
`fixed32`	Always four bytes. More efficient than uint32 if values are often greater than 228.	`uint32`	`int`	`int`
`fixed64`	Always eight bytes. More efficient than uint64 if values are often greater than 256.	`uint64`	`long`	`int/long`
`sfixed32`	Always four bytes.	`int32`	`int`	`int`
`sfixed64`	Always eight bytes.	`int64`	`long`	`int/long`
`bool`	bool	`boolean`	`bool`	`bool`
`string`	A string must always contain UTF-8 encoded or 7-bit ASCII text.	`string`	`String`	`str/unicode`
`bytes`	May contain any arbitrary sequence of bytes.	`string`	`ByteString`	`str`

Mensajes

- Los mensajes describen la información que se enviará como petición y la que se recibirá como respuesta
- Pueden contener varios campos de diferentes tipos
- Se permite herencia (extensión)
- Por ello a cada campo se le asocia un número

```
message SearchRequest {
  string query = 1;
  int32 page_number = 2;
  int32 results_per_page = 3;
}
```

- Los campos pueden ir precedidos de `repeated` para indicar que se pueden repetir **cero o más veces**

Enumerados

- Se pueden incluir también enumerados:

```
enum Corpus {
    UNIVERSAL = 0;
    WEB = 1;
    IMAGES = 2;
    LOCAL = 3;
    NEWS = 4;
    PRODUCTS = 5;
    VIDEO = 6;
  }
```

- Y luego ser usados como parte de mensajes:

```
message XX {
    // ...
    Corpus corpus = 4;
}
```

Mensajes compuestos

- Los tipos de los campos de los mensajes pueden incluir otros mensajes definidos previamente:

```
message SearchResponse {
  repeated Result results = 1;
}

message Result {
  string url = 1;
  string title = 2;
  repeated string snippets = 3;
}
```

- Y se pueden importar definiciones de otros ficheros con `import "file.proto"`

Tipo Any

- Se puede definir un tipo genérico (como en IDL de CORBA) que guarde cualquier otro tipo
- Es el tipo **Any**:

```
import "google/protobuf/any.proto";

message ErrorStatus {
  string message = 1;
  repeated google.protobuf.Any details = 2;
}
```

- Después se verá el *mapping* a Java

Oneof

- Se puede definir campos excluyentes:

```
message SampleMessage {
  oneof test_oneof {
    string name = 4;
    SubMessage sub_message = 9;
  }
}
```

- El mapping incluye funciones **set_** para cada uno de los atributos,
- Y también funciones para comprobar si lo tienen **has_**

Mapas

- Se pueden definir tipos mapa:

```
map<string, Project> projects = 3;
```

- La clave puede ser cualquier tipo escalar salvo `float` y bytes

Paquetes

- Similar a como se puede especificar en Java o en C++, se permite definir paquetes y referencia a otros mensajes definidos en otros paquetes:

```
package foo.bar;
message Open { ... }

// ...

message Foo {
  // ...
  foo.bar.Open open = 1;
  // ...
}
```

Servicios

- Finalmente, se pueden definir servicios (el equivalente a los interfaces en Java)
- Se utiliza la palabra clave **service** y cada llamada con **rpc**:

```
// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings
message HelloReply {
  string message = 1;
}

// The greeting service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}
```

Compilación de los ficheros `.proto`

- Para comenzar, los ficheros `.proto` se tienen que convertir a código Java:
  - Permite abrir canales de comunicación
  - Realizar peticiones y recoger respuestas
  - Y también manejar objetos de los tipos definidos por los `message`s
- La compilación de los ficheros `.proto` a Java se puede realizar de dos formas:
  1. Usando la utilidad `protoc`:

     ```
     $ protoc -I=$SRCDIR --java_out=$DSTDIR $SRCDIR/fichero.proto
     ```

     y después la utilidad `protoc-gen-grpc-java`

Compilación de los ficheros `.proto` II

2. Usando **Maven** (para esto hay que especificar en el fichero `pom.xml` que se quiere usar el plugin `protobuf-maven-plugin`).
   El ejemplo de cómo especificarlo en el fichero `pom.xml` del proyecto se puede ver en el proyecto de ejemplo `grpc-test1`. Para generarlo:

   ```
   $ mvn generate-sources
   ```

   o bien:

   ```
   $ mvn compile
   ```

Mapping de mensajes

- Para cada mensaje se genera una clase con el mismo nombre
- Para cada campo del mensaje se generan funciones para obtener y modificar ese dato
- gRPC utiliza el patrón *Builder* para construir objetos, mensajes, canales de comunicación, etc.

Mapping de mensajes II

- Para crear un objeto de un mensaje **Mensage**, se utiliza la función **newBuilder()**, y después se pueden usar las funciones de establecer campos. Finalmente, debe terminar con **.build()**:

```
Mensaje m = Mensaje.newBuilder()
                    .setCampoX(valor)
                    .setCampoY(valor)
                    .build();
```

Mapping de mensajes III

- Por ejemplo, dado el siguiente `.proto`:

```
message Person {
  string name = 1;
  int32 id = 2;
  string email = 3;

  enum PhoneType {
    MOBILE = 0;
    HOME = 1;
    WORK = 2;
  }

  message PhoneNumber {
    string number = 1;
    PhoneType type = 2 [default = HOME];
  }

  repeated PhoneNumber phone = 4;
}
```

Mapping de mensajes IV

- Se genera la clase **Person** con unos métodos para acceso a los campos:

```
// string name = 1;
public boolean hasName();
public String getName();

// int32 id = 2;
public boolean hasId();
public int getId();

// string email = 3;
public boolean hasEmail();
public String getEmail();

// repeated Person.PhoneNumber phone = 4;
public List<PhoneNumber> getPhoneList();
public int getPhoneCount();
public PhoneNumber getPhone(int index);
```

Mapping de mensajes V

- Por su parte el *builder* tiene:

```
// string name = 1;
public boolean hasName();
public java.lang.String getName();
public Builder setName(String value);
public Builder clearName();

// int32 id = 2;
public boolean hasId();
public int getId();
public Builder setId(int value);
public Builder clearId();

// string email = 3;
public boolean hasEmail();
public String getEmail();
public Builder setEmail(String value);
public Builder clearEmail();
```

Mapping de mensajes VI

```
// repeated .tutorial.Person.PhoneNumber phone = 4;
public List<PhoneNumber> getPhoneList();
public int getPhoneCount();
public PhoneNumber getPhone(int index);
public Builder setPhone(int index, PhoneNumber value);
public Builder addPhone(PhoneNumber value);
public Builder addAllPhone(Iterable<PhoneNumber> value);
public Builder clearPhone();
```

- Contiene métodos adicionales para ir construyendo el objeto

Mapping de mensajes VII

- Por su parte, para construir un mensaje:

```
Person john =
  Person.newBuilder()
    .setId(1234)
    .setName("John Doe")
    .setEmail("jdoe@example.com")
    .addPhone(
      Person.PhoneNumber.newBuilder()
        .setNumber("555-4321")
        .setType(Person.PhoneType.HOME).build())
    .build();
```

Mapping de mensajes VIII

- El caso especial de los mapas:

```
map<int32, int32> weight = 1;
```

- Genera en la clase del mensaje:
  - `Map<Integer, Integer> getWeightMap();` – Retorna un mapa no modificable
  - `int getWeightOrDefault(int key, int default);` – Retorna el valor para la clave dada, o el valor por defecto si no existe
  - `int getWeightOrThrow(int key);` – Retorna el valor para la clave dada o si no lanza la excepción `IllegalArgumentException`
  - `boolean containsWeight(int key);` – Indica si está pendiente el campo
  - `int getWeightCount();` – Retorna el número de elementos del mapa

Mapping de mensajes IX

- Y en el *builder*:
  - `Builder putWeight(int key, int value);` – Añade el valor asociado a la clave
  - `Builder putAllWeight(Map<Integer, Integer> value);` – Añade todos los elementos a *weight*
  - `Builder removeWeight(int key);` – Elimina esa clave del campo
  - `Builder clearWeight();` – Elimina todas las claves para este campo

Mapping de mensajes X

- Tipo `com.google.protobuf.Any`:

```
class Any {
  // Packs the given message into an Any using the default type URL
  // prefix "type.googleapis.com".
  public static Any pack(Message message);
  // Packs the given message into an Any using the given type URL
  // prefix.
  public static Any pack(Message message,
                         String typeUrlPrefix);

  // Checks whether this Any message's payload is the given type.
  public <T extends Message> boolean is(class<T> clazz);

  // Unpacks Any into the given message type. Throws exception if
  // the type doesn't match or parsing the payload has failed.
  public <T extends Message> T unpack(class<T> clazz)
      throws InvalidProtocolBufferException;
}
```

Servicios

- Los servicios se especifican con la palabra clave `service`
- Especifican diferentes mensajes definidos con la palabra clave `rpc`
- Hay dos tipos de llamadas:
  - **Síncrona**: RPC clásico, se realiza una petición y se espera una respuesta
  - **Asíncrona**: Se realiza una petición y no se espera la respuesta. Se puede comprobar después si la respuesta ha llegado
- Las llamadas también pueden especificar que envían y/o reciben un *stream* de datos, en vez de un solo dato
  - Aumenta la eficiencia al enmascarar la **latencia**

RPC estándar

- Un servicio de construcción de *collages* de imágenes:

```
package collage;

service Collage {
  rpc StoreImage(ImageData) returns (google.protobuf.Empty);
  rpc StoreImages(stream ImageData) returns (google.protobuf.Empty);
  rpc ObtainImage (ImageSpec) returns (ImageData);
  rpc ObtainCollage (stream ImageSpec) returns (ImageData);
}

message ImageData{
        string id = 1;
        bytes data = 2;
}

message ImageSpec {
        string id = 1;
}
```

RPC estándar II

- La llamada **`StoreImage`** es una llamada RPC normal, donde el cliente envía una imagen a almacenar y el servidor no responde nada
- El cliente construye la llamada normalmente:

```
ImageData image = ImageData.newBuilder().setId(/*datos imagen1*/"imagen1")
    .setData(ByteString.copyFrom("Imagen 1 data".getBytes())).build();
stub.storeImage(image);
```

  (aquí **`stub`** especifica el objeto al que el cliente puede llamar en local y que causará la llamada remota al servidor)
- El **servidor** tiene una clase, que llamaremos **`CollageImpl`**, que hereda de **`CollageGrpc.CollageImplBase`** (generada por el compilador de gRPC)

RPC estándar III

- Esta clase implementa todos los métodos del servicio:

```
class CollageImpl extends CollageGrpc.CollageImplBase
{
  private Map<String, ImageData> imageMap;

  @Override
  public void storeImage(ImageData request, StreamObserver<Empty>
      responseObserver)
  {
    logger.info("Add image {}.".format(request.getId()));
    imageMap.put(request.getId(),request);
    responseObserver.onNext(Empty.newBuilder().build());
    responseObserver.onCompleted();
  }
```

RPC estándar IV

- Las peticiones normales reciben:
  - Los argumentos (en este caso `ImageData request`)
  - Un `StreamObserver<TipoRetorno>`, que permite enviar varias respuestas al cliente
  - Esto permite que se pueda enviar un `stream` de respuesta sin tener que cambiar la signatura
- El interfaz `StreamObserver<>` tiene los métodos `onNext()` (para ir enviando elementos) y `onCompleted()` (cuando se finaliza)

RPC *streaming*/asíncrono (cliente)

- Algunas combinaciones de *stream* no pueden realizarse con el *stub* bloqueante
- Hay que usar el *stub* asíncrono
- Veremos la llamada `storeImages(stream ImageData) returns Empty`
- Como no retorna nada, se puede dejar la llamada sin esperar la respuesta, pero se verá como ejemplo
- Hay que decidir un tiempo prudencial (más de esto después)

RPC *streaming*/asíncrono (cliente) II

- La creación del *stub* asíncrono se realiza igual que el bloqueante, pero con **newStub()** en vez de **newBlockingStub()**
- Por ejemplo, para el caso de las imágenes:

```
ManagedChannel channel = ManagedChannelBuilder.forAddress(host, port)
    .usePlaintext()
    .build();

private final CollageGrpc.CollageStub asyncStub;
asyncStub = CollageGrpc.newStub(channel);
```

RPC *streaming*/asíncrono (cliente) III

- Se tiene que crear un *Latch* (un punto de sincronización)
- Normalmente un `CountDownLatch`. La llamada puede terminar:
  1. Si el contador llega a 0 (`onCompleted()` u `onError()`)
  2. Si el temporizador llega a 0. En este caso se tiene que comprobar si se ha recibido la respuesta y actuar en consecuencia

RPC *streaming*/asíncrono (cliente) IV

```java
try {
  final CountDownLatch finishLatch = new CountDownLatch(1);
  StreamObserver<Empty> soEmpty = new StreamObserver<Empty>() {
          @Override
          public void onNext(Empty value) { }
          @Override
          public void onError(Throwable t) {
                  finishLatch.countDown(); // !!
          }
          @Override
          public void onCompleted() {
                  finishLatch.countDown(); // !!
          }
  };

  // Se envían las imágenes una a una
  StreamObserver<ImageData> so = asyncStub.storeImages(soEmpty);
  so.onNext(image1);
  so.onNext(image2);
  so.onCompleted();

  // Esperar la respuesta
  if (finishLatch.await(1, TimeUnit.SECONDS))
          logger.info("Received response.");
  else
          logger.info("Not received response!");
} catch (StatusRuntimeException e) {
```

RPC *streaming*/asíncrono (cliente) V

- En este caso, se espera un segundo a recibir la respuesta
- Mientras tanto el `StreamObserver` (`soEmpty`) puede ir recibiendo las respuestas del servidor (si las hubiera, no hay en este caso) y al recibir la finalización hace terminal el *latch* de forma correcta

RPC *streaming*/asíncrono (servidor)

- En el servidor **la signatura del método `storeImages()` debe ser la misma que en el cliente**
- Por lo que debe retornar también un `StreamObserver<ImageData>`
- En este caso, será el *stub* de servidor el que usará el objeto para ir llamando al método `onNext()` con cada parámetro enviado por el cliente

RPC *streaming*/asíncrono (servidor) II

En la clase **CollageImpl**:

```
public StreamObserver<ImageData> storeImages(StreamObserver<Empty> responseObserver)
{
    // Respuesta (empty)
    responseObserver.onNext(Empty.newBuilder().build());

    // Se retorna un objeto que, al ser llamado en onNext() con cada
    // elemento enviado por el cliente, reacciona correctamente
    return new StreamObserver<ImageData>() {
        @Override
        public void onCompleted() {              // Terminar la respuesta
            responseObserver.onCompleted();
        }
        @Override
        public void onError(Throwable arg0) {
        }
        @Override
        public void onNext(ImageData imagedata)
        {
            // Añadir la imagen
            logger.info("Add image (multiple) " + imagedata.getId());
            imageMap.put(imagedata.getId(), imagedata);
        }
    };
}
```

Creación del servidor

- Se tiene que crear alguna aplicación servidora que se quede a la espera de clientes del servicio
- Existen las clases **Server** y **ServerBuilder**
- Nótese cómo se añade el servicio **CollageImpl**

```
int port = // PORT
Server server;

// ...

server = ServerBuilder.forPort(port)
        .addService(new CollageImpl(logger))
        .build()
        .start();
```

Creación del cliente

- Cualquier cliente que quiera acceder al servidor tiene que crear un "canal"
- Se tienen las clases **ManagedChannel** y su correspondiente *builder*
- Los *stubs* se pueden usar para llamar al servicio

```
private final ManagedChannel channel;
private final CollageGrpc.CollageBlockingStub blockingStub;
private final CollageGrpc.CollageStub asyncStub;

channel = ManagedChannelBuilder.forAddress(host, port)
    // Channels are secure by default (via SSL/TLS).
    // For the example we disable TLS to avoid
    // needing certificates.
    .usePlaintext()
    .build();
  blockingStub = CollageGrpc.newBlockingStub(channel);
  asyncStub = CollageGrpc.newStub(channel);
```

Versionado de APIs

- Un reto importante en sistemas distribuidos es mantener compatibilidad con versiones antiguas
- En gRPC, los cambios incompatibles rompen clientes antiguos

Versionado de APIs II

- **Problema**: Si cambiamos la respuesta, clientes antiguos fallan

```
// Versión 1 (original)
service UserService {
  rpc GetUser(UserId) returns (User);
}

message User {
  int32 id = 1;
  string name = 2;
}

// Versión 2 (cambio incompatible)
message UserProfile {          // Nombre diferente
  int32 id = 1;
  string name = 2;
  string email = 3;            // Campo nuevo
}
```

Versionado de APIs III

Solución: Usar versionado en paquetes y proporcionar métodos compatibles

```
// user/v1/user.proto
package user;

message User {
  int32 id = 1;
  string name = 2;
}

service UserService {
  rpc GetUser(UserId) returns (User);
}
```

Versionado de APIs IV

```
// user/v2/user.proto
package user.v2;

message UserProfile {
  int32 id = 1;
  string name = 2;
  string email = 3;
  bool is_active = 4;
}

service UserService {
  rpc GetUser(UserId) returns (user.User);
  // Mantener soporte para clientes v1
  rpc GetUserV2(UserId) returns (UserProfile);
}
```

Versionado de APIs V

Los clientes pueden escoger qué versión usar:

```
// Cliente antiguo (usa v1)
user.UserServiceGrpc.UserServiceBlockingStub clientV1 =
  user.UserServiceGrpc.newBlockingStub(channel);
user.User user = clientV1.getUser(id);

// Cliente nuevo (usa v2)
user.v2.UserServiceGrpc.UserServiceBlockingStub clientV2 =
  user.v2.UserServiceGrpc.newBlockingStub(channel);
user.v2.UserProfile profile = clientV2.getUser(id);
```

También se permite agregar nuevos campos sin problemas:

- Clientes antiguos los ignoran automáticamente
- Es la forma segura de evolucionar APIs

Gestión de números de campos

- Cada campo en un mensaje tiene asignado un número único
- **No deben reutilizarse números** de campos borrados
- Puede causar serialización incorrecta

Gestión de números de campos II

```
// MALO: reutilización de número
message Product {
  int32 id = 1;
  string name = 2;
  // string description = 3;  // Borrado
  bool is_available = 3;      // ERROR: mismo número!
}
// BIEN: reservar números obsoletos
message Product {
  int32 id = 1;
  string name = 2;
  bool is_available = 4;      // Número diferente

  // Reservar para evitar reutilización accidental
  reserved 3;
  reserved "description";
}
```

- El compilador *protoc* protesta si reutilizas un número reservado

Ejemplo: Transferencia de ficheros por streaming

- Transferir ficheros grandes en un único mensaje es ineficiente
- Solución: dividir en ***chunks*** (trozos)
- El cliente envía el fichero en segmentos pequeños

```
// file.proto
message FileChunk {
  string filename = 1;
  int32 chunk_number = 2;
  bytes data = 3;                    // Máximo ~1MB
  int32 total_chunks = 4;
}

message UploadStatus {
  bool success = 1;
  int32 bytes_received = 2;
  string message = 3;
}

service FileService {
  rpc UploadFile(stream FileChunk) returns (UploadStatus);
}
```

Ficheros por streaming

- Características importantes:
  - **Validación de orden**: Los chunks deben llegar en secuencia
  - **Manejo de errores**: Se cierra el fichero en caso de error
  - **Confirmación**: El servidor informa de bytes recibidos
  - **Escalabilidad**: Permite transferir ficheros de tamaño arbitrario sin cargar la memoria

Buenas prácticas adicionales

- **Tamaño máximo de mensajes**: Configurar para evitar problemas de memoria
  - Por defecto gRPC limita a 4MB
  - Pueden aumentarse pero es arriesgado
- **Reutilización de conexiones**: Mantener canales abiertos en el cliente, no crear nuevos por cada petición
- **Timeouts y Deadlines**: Siempre configurar para evitar clientes/servidores que se quedan esperando indefinidamente
- **Validación de datos**: Validar en servidor siempre
- **Logging y monitoreo**: Usar interceptores para auditoría de llamadas y métricas
- **Manejo de conexiones**: Implementar `shutdown()` y `awaitTermination()` correctamente